UNIT-4

Discovering Patterns and Rules
Frequent Pattern Mining
Basic Concepts and a Road Map
Efficient and scalable frequent item set mining methods
Apriori algorithm
FP-Growth algorithm
Mining frequent itemsets using vertical data format
Mining closed and max patterns-
Advanced Pattern Mining:
Pattern Mining in Multilevel, Multidimensional Space

Data mining is the process of transforming huge volumes of raw data into useful information and gaining meaningful insights. It has nothing to do with the extraction/assimilation of data, in fact, existing data volumes are its input only. It's done with a clearly defined purpose in mind, and its end result is to present its findings in an easily comprehensible manner, such that it can aid decision-making. Under the Data Mining hood, various parallel activities like Data Management, Pre-processing, Data Warehousing, Pipelining and loading of data for a particular purpose, creation of Machine-Learning/Artificial Intelligence models, analyzing the data via these models and existing statistical methods, Pattern Discovery and Pattern Analysis, and finally visualizing the facts uncovered, etc. are performed.

Probably, Knowledge Discovery in Data (KDD) or Data Analysis would be a more appropriate term for it

Why is it Important to Recognize a Pattern?

Apart from the usual mathematical calculations like totals/correlations/variations/ratios/derivatives etc., there is a lot more information that is lying covered inside your data. A pattern could show an emerging trend, a receding factor, ups and downs of a certain behavior, seasonal variations, etc., amongst many other facts.

A pattern generally shows that there exists some useful information waiting to be unearthed from the data. Here, we will discuss patterns as they occur in Data Science and some intricacies of pattern discovery in data mining.

What is Pattern Discovery in Data Mining?

A pattern is something that repeats itself given a set of conditions or a set of rules. Recognizing patterns leads to useful information, eases decision making, and makes our tasks simpler.

Some of the different types of patterns that one could see in data are:-

- **Periodic Patterns**: These are patterns that are seen repeating themselves after a certain lapse of time. This type of pattern is frequently encountered in time series data, biological sequences, spatiotemporal (has both time + space dimensions) data, etc.
- **Associative Patterns**: These are like Bread and Butter/Knife and Fork, i.e. co-occurring groups of things that make more sense with each other. The members of this pattern are complementary to each other.
- **Abnormal Patterns:** This kind of pattern occurs when the data has a clear deviation from normal behavior, an unexpected pattern appears between expected patterns and its appearance is not periodic.
- **Structural Patterns**: Like path finding in graphs or cluster identification > An example would be low-cost residences tend to occur in suburbs whereas downtown has higher costing apartments.

Chaotic Patterns: Here the patterns appear but have no definitive characteristic related to time/space/frequency.

Where do Pattern Discovery and Pattern Recognition find their Major Applications?

Pattern Discovery in Data Mining and analysis are very useful in:

- **Sentiment and Trend Analysis**: Here patterns can give you insights into the sentiments or mood of your customers/investors. Whether the sentiment is upswing or positive, which could lead to an increase in demand and price, or if it's negative. Also, you can uncover trends like if something is being liked by the masses or getting unpopular. This application has widespread usage in e-commerce, the retail industry, investment banking, voting patterns, etc.
- **Image Processing & Computer vision:** A computer is imparted human-like skills to recognize things by processing their outward features, orientations, and actions.
- **Bioinformatics:** Large and complex biological datasets can be processed to know the side effects of drugs, the onset of a pandemic, deficiencies of a population, genetic basis of diseases, or changes at the molecular/cellular level.
- **Forensic Analysis and DNA Sequencing:** Knowing the biological parents of someone or identifying a culprit by remnants of his hair/sputum/skin etc.
- **Forecasting and Probabilistic Estimation:** Weather forecasting, forecasting changes in consumer behavior, estimating if a new product would get popular with a subset of the target population.
- **Assistance and Support Automation:** Assisting people and providing better support via the use of technology, automating redundant daily tasks to focus more on the crucial ones thereby increasing employee productivity.
- **Seismic Analysis:** Analyzing if a building will stay safe during earthquakes/hurricanes, providing structures with seismic stability.
- **Speech Recognition:** As the name suggests, it's about recognizing the speakers amongst a set of preloaded voices, deciphering the message/intent the speaker wishes to convey, etc.

Pattern Discovery in Data Mining can be done via various techniques and tools.

What are Some Methods of Pattern Discovery in Data?

Though a human eye can detect patterns when the data is small, the huge volume of data in question needs to be handled differently. Some of the prevalent methods of Pattern Discovery in Data Mining are:

1) Through Self-developed Customized Programs

If your dataset is not huge or your pattern recognition requirements are simple, you can consider developing your own custom program to run through the data and perform Pattern Discovery in Data Mining. This way you could have a lot of flexibility and customizability built into your program, to suit your specific requirements.

This approach also provides you with total control over your pattern discovery process, and you're not constrained by the capabilities of a third-party tool. Most of today's programming languages have mature existing libraries to aid you in pattern detection.

E.g. Python has **PyTorch** for Deep Learning and **OpenCV** for Computer Vision, Java has **JADEPT** (Java Design Pattern detector), and Tesseract OCR, JavaScript has node-Tesseract and etc.

2) Through Machine Learning

In Machine Learning, you first create a **Machine Learning Model.** A model here is a program that is trained to recognize patterns or similarities in behavior. While creating a model, you fill it with a set of instructions, similar to a program, and then you train it on sample data.

E.g. You can train a model to recognize a boat in a picture, akin to what you see in captchas.

So, you can train the model by first making it learn to recognize a boat shape. This will include some programmatic instructions to measure/gauge the shapes and contours of objects and then recognize a boat-shaped structure.

Next, you train that model by feeding it pictures that contain a boat and a few other objects within the same image. After some practice, the model becomes fairly perfect in recognizing boats inside pictures.

Now, this model can be used to detect all images that contain boat(s) from a stream of images. Similarly, a trained Machine Learning Model is used to detect patterns/trends/similarities in behaviour, while reading a stream of incoming data.

Now, Who Feeds the Model with the Data?

It's your Machine Learning Algorithm that first loads and pre-processes data, and then feeds it to the trained model. So another important aspect here is the algorithm.

You can create algorithms that use trained models to recognize Linear Regression, classify entities based on detected characteristics or cluster similar objects together.

In practice, you will have a set of Machine Learning Algorithms, packaged as a library or provided as an API, to be used for pattern discovery as well as many other functions.

What makes Hevo's ETL Process Best-In-Class

Providing a high-quality ETL solution can be a difficult task if you have a large volume of data. Hevo's automated, No-code platform empowers you with everything you need to have for a smooth data replication experience.

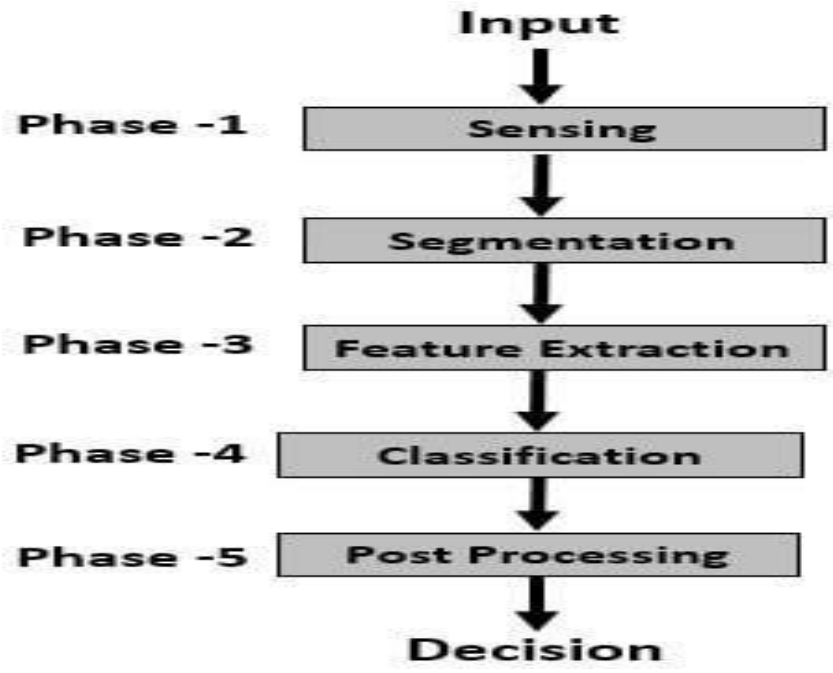
Check out what makes Hevo amazing:

- **Fully Managed**: Hevo requires no management and maintenance as it is a fully automated platform.
- **Data Transformation**: Hevo provides a simple interface to perfect, modify, and enrich the data you want to transfer.
- **Faster Insight Generation**: Hevo offers near real-time data replication so you have access to real-time insight generation and faster decision making.
- **Schema Management**: Hevo can automatically detect the schema of the incoming data and map it to the destination schema.
- **Scalable Infrastructure**: Hevo has in-built integrations for 100+ sources (with 40+ free sources) that can help you scale your data infrastructure as required.
- **Live Support**: Hevo team is available round the clock to extend exceptional support to its customers through chat, email, and support calls.

SIGN UP HERE FOR A 14-DAY FREE TRIAL!

What are the Major Steps that are executed in the Process?

Pattern Discovery in Data Mining via Machine Learning (ML) can be broken into these major steps:



Step 1: Sensing Data

The input data can be **sporadic/irregular/chunky**, so in this phase, the Pattern Recognition system smoothens the input data into continuous/analogous data.

Step 2: Data Segmentation

In this phase, the system will try to eliminate the noise and isolate the useful/sensed objects.

Step 3: Feature Extraction

This phase takes the pruned data as input, all the useful objects are further processed and their features/properties are identified, and then the data is sent for further classification.

Step 4: Classification

In this phase, the sensed objects are grouped/classified based on their properties. The data is now ready to be fed to the model/rule system.

Step 5: Post-processing

Here, further considerations are made before a final Pattern Discovery in Data Mining or pattern recognition is made.

What are the Problems Encountered during Pattern Discovery in Data Mining?

One could encounter problems and false positives while using Pattern Discovery in Data Mining. Some of these could be:

- Since Data Mining tools are generally not good at detecting **poor quality data**, this poor quality data if it exists may lead to false patterns or improbable patterns. So before concluding that there exists a useful pattern, one must make sure that the data that the pattern emerges from, is of good quality.
- **Some patterns are quite obvious** even without Data Mining; such patterns should be kept aside while arriving at any conclusion.
- Some patterns will not pass the **test of time**, e.g. buying patterns of dwellers of a village are of no use if the population in question migrated away from that area due to some reason.
- Compare carefully and generalize sparingly – **what are the odds that someone will survive being hit by 2 gunshots?**

    - The probability of someone surviving a single gunshot is very low.
    - But the probability of someone surviving 2 gunshots is close to even when you compare it with people who have never received a gunshot. Because to calculate this probability you will have to include those people who have already survived 1 gunshot.

- Hence, a highly improbable event might seem plausible if compared with the inappropriate fact.

- At times you will need to discern that though a pattern exists, it's not pointing to any useful information or does not represent something relevant to your purpose.

Conclusion

Pattern Discovery in Data Mining improves the efficiency and quality of Insights. Patterns help in understanding the data better, as well as provide more meaning to data. Pattern Discovery in Data Mining is a 5 step process that is discussed in detail. To conclude, we have discussed Pattern Discovery in Data Mining in simple terms, and how to make the best use of it, in your Big Data Analytics practice.

There are various Data Sources that organizations leverage to capture a variety of valuable data points. But, transferring data from these sources into a Data Warehouse for a holistic analysis is a hectic task. It requires you to code and maintains complex functions that can help achieve a smooth flow of data. Machine learning algorithms are now synonymous with finding patterns in data but not all patterns are suitable for statistics based data-driven techniques, for example when these patterns don't have explicitly labeled targets to learn from. In some cases, these patterns can be expressed precisely as a rule. Reasoning is the process of matching rule-based patterns or verifying that they don't exist in a graph. Because these patterns are found with deductive logic they can be found more efficiently and interpreted more easily than Machine Learning patterns which are induced from the data.

Frequent Pattern Mining in Data Mining

The increasing power of computer technology creates a large amount of data and storage. Databases are increasing rapidly and in this computerized world everything is shifting online and data are increasing as a new currency. Data comes in different shapes and sizes and is collected in different ways. By using data mining there are many benefits it helps us to improve the particular process and in some cases, it costs saving or revenue generation. Data mining is commonly used to search a large amount of data for patterns and trends, and not only for searching it uses the data for further processes and develops actionable processes.

Data mining is the process of converting raw data into suitable patterns based on trends.

Data mining has different types of patterns and **frequent pattern mining** is one of them. This concept was introduced for mining transaction databases. Frequent patterns are patterns (such as items, subsequences, or substructures) that appear frequently in the database. It is an analytical process that finds frequent patterns, associations, or causal structures from databases in various databases. This process aims to find the frequently occurring item in a transaction. By frequent patterns, we can identify strongly correlated items together and we can identify

similar characteristics and associations among them. By doing frequent data mining we can go further for clustering and association.

Frequent pattern mining is a major concern it plays a major role in associations and correlations and discloses an intrinsic and important property of dataset.

Frequent data mining can be done by using association rules with particular algorithms éclat and apriori algorithms. Frequent pattern mining searches for recurring relationships in a data set. It also helps to find the inheritance regularities. To make fast processing software with a user interface and used for a long time without any error.

Association Rule Mining:

It is easy to find associations in frequent patterns:

- For each frequent pattern x for each subset y c x.
- Calculate the support of y-> x – y.

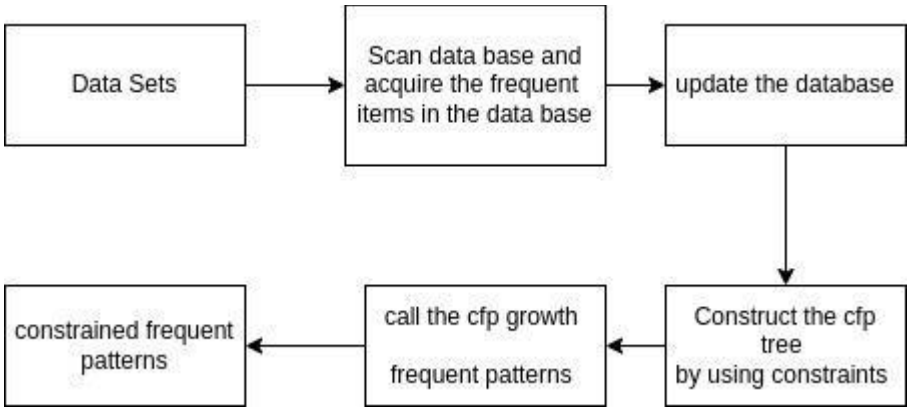
If it is greater than the threshold, keep the rule. There are two algorithms that support this lattice

1. Apriori algorithm
2. éclat algorithm

Apriori	Éclat
It performs "perfect" pruning of infrequent item sets.	It reduces memory requirements and is faster.
It requires a lot of memory (all frequent item sets are represented) and support counting takes very long for large transactions. But this is not efficient in practice.	Its storage of transaction list.

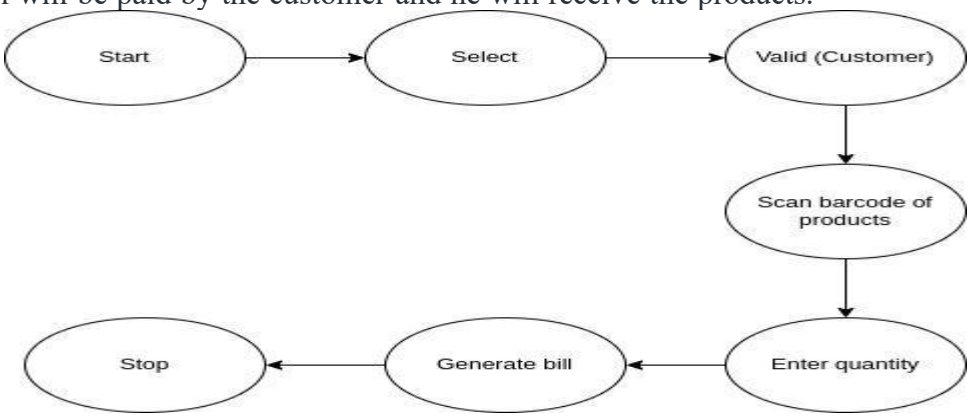
The words support and confidence support the association rule.

- **Support:** how often a given rule in a database is mined? support the transaction contains x U y
- **Confidence:** the number of times the given rule in a practice is true. The conditional probability is a transaction having x as well as y.



Working principle (it is a simple point of scale application for any supermarket which has a good off-product scale)

- The product data will be entered into the database.
- The taxes and commissions are entered.
- The product will be purchased and it will be sent to the bill counter.
- The bill calculating operator will check the product with the bar code machine it will check and match the product in the database and then it will show the information of the product.
- The bill will be paid by the customer and he will receive the products.



Tasks in the frequent pattern mining:

- Association
- **Cluster analysis:** frequent pattern-based clustering is well suited for high-dimensional data. By the extension of dimension the sub-space clustering occurs.
- **Data warehouse:** iceberg cube and cube gradient
- Broad applications

There are some to improve the efficiency of the tasks.

Closed Pattern:

A frequent pattern, it meets the minimum support criteria. All super patterns of a closed pattern are less frequent than the closed pattern.

Max Pattern:

It also meets the minimum support criteria (like a closed pattern). All super patterns of a max pattern are not frequent patterns. Both patterns generate fewer numbers of patterns so therefore they increase the efficiency of the task.

Applications of Frequent Pattern Mining:

Basket data analysis, cross-marketing, catalog design, sale campaign analysis, web log analysis, and DNA sequence analysis.

Issues of frequent pattern mining

- flexibility and reusability for creating frequent patterns
- most of the algorithms used for mining frequent item sets do not offer flexibility for rescuing
- much research is needed to reduce the size of the derived patterns

Conclusion:

It is impossible to give complete coverage of this topic with the limited space and our limited knowledge. Frequent pattern mining has achieved tremendous progress and claimed a good set of applications. However in-depth research is required that the field may have a long-lasting and deep impact on data mining applications.

In Data Mining, Frequent Pattern Mining is a major concern because it is playing a major role in Associations and Correlations. First of all, we should know what is a Frequent Pattern? Before moving to mine frequent patterns, we should focus on two terms which “support” and “confidence” because they can provide a measure if the Association rule is qualified or not for a particular data set.

Support: how often a given rule appears in the database being mined

Confidence: the number of times a given rule turns out to be true in practice

Let's practice it through a sample data set;

Transaction	Item Occurrence
T1	A, B, C
T2	A, C, D
T3	A, B, C, D
T4	A, D, E
T5	B, C

Image by Author

$$\text{Support} = \text{Frequency}(A, B) / N$$

Support calculation — Image by Author

$$\text{Confidence} = \text{Frequency}(A, B) / \text{Frequency}(A)$$

Confidence calculation — Image by Author

Example: One of possible Association Rule is A => D

Total no of Transactions (N) = 5

Frequency (A, D) = > Total no of instances together A with D is 3

Frequency (A) => Total no of occurrence in A

Support = 3 / 5

Confidence = 3 / 4

After getting a clear idea about the two terms Support and Confidence, we can move to frequent pattern mining. Frequent pattern mining, there are 2 categories to be considered,

1. Mining frequent pattern with candidate generation
2. Mining frequent pattern without candidate generation

In this article, we are focusing on Mining frequent patterns with candidate generation with Apriori Algorithm which is popularly used for Association mining. Let's understand Apriori Algorithm with an example and it will help you to understand the concept behind it in a clear manner. Let's consider the sample data set mentioned above and assume that the **minimum support=2**.

1. **Generate Candidate set 1, do the first scan and generate One item set**

In this stage, we get the sample data set and take each individual's count and make frequent item set 1(K = 1).

Item	Support Count
A	4
B	3
C	3
D	3
E	1

Candidate set 1

“Candidate set 1” figure shows you the individual item Support count. Hence the minimum support is 2 and based on that, item E will remove from the Candidate set 1 as an infrequent item (disqualified).

Item	Support Count
A	4
B	3
C	3
D	3

Frequent item set from the first scan Frequent item set based on the minimum support value will be shown under the figure “Frequent item set from the first scan” as the “**One item set**”.

2. **Generate Candidate set 2, do the second scan and generate Second item set**

Through this step, you create frequent set 2 (K =2) and takes each of their Support counts.

Item	Support Count
A,B	2
A,C	2
A,D	3
B,C	2
B,D	1
C,D	2

Candidate set 2

“Candidate set 2” figure generates through joining Candidate set 1 and takes the frequency of related occurrences. Hence the minimum support is 2, Item set B, D will be removed from Candidate set 2 as an infrequent item set.

Item	Support Count
A,B	2
A,C	2
A,D	3
B,C	2
C,D	2

Frequent item set from the second scan

"Frequent item set from the second scan" is the frequent itemset based on the minimum support value and it will generate the "**Second item set**".

3. Generate Candidate set 3, do the third scan and generate Third item set

In this iteration create frequent set 3 (K = 3) and take count of Support. Then compare with the minimum support value from the Candidate set 3.

Item	Support Count
A,B,C	2
A,C,D	2

Candidate set 3

As you can see here we compare Candidate set 3 with the minimum support value, generated Third item set. The frequent set from the third scan is the same as above.

4. Generate Candidate set 4, do the fourth scan and generate Fourth item set

By considering the Frequent set, we can generate Candidate set 4, by joining Candidate set 3. Then possible Candidate set 4 is; [A, B, C, D] which has minimum support less than 2. Therefore we have to stop the calculation from here because we can not go no any more iterations. Therefore for the above data set frequent patterns are **[A, B, C]** and **[A, C, D]**.

By considering one of the frequent set as{A, B, C} and the possible Association rules as follows;

1. A => B, C
2. A, B => C
3. A, C => B
4. B => A, C
5. B, C => A
6. C => A, B

Then we assume that the minimum confidence = 50% and calculate each possible Association rules' Confidence then we can identify disqualified ones with respect to minimum confidence is less than 50%. Then rest of the Association rules which have the Confidence greater or equal to 50% percent those are qualified rules.

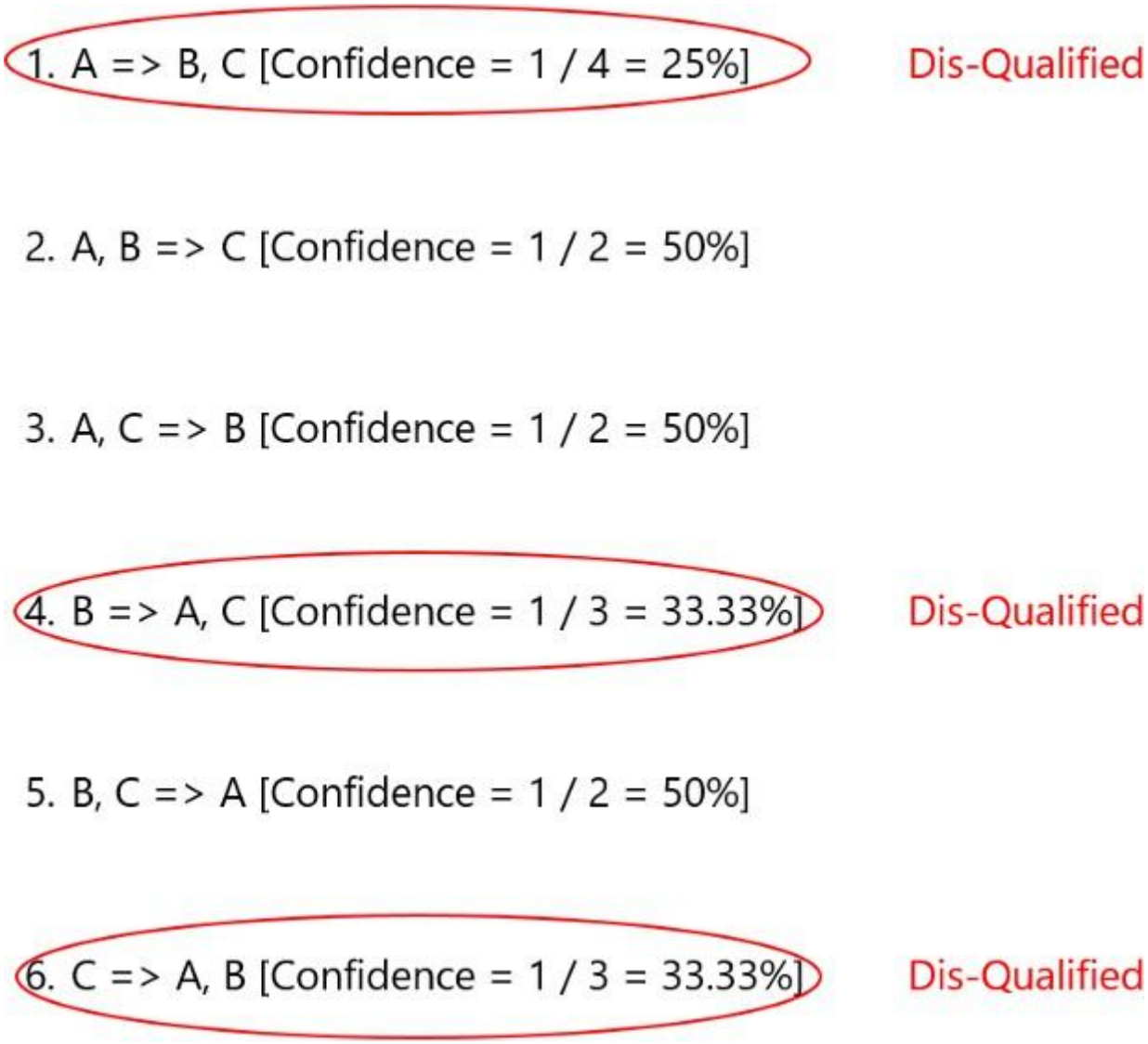


Image by Author

As you can see in the Apriori Algorithm, for each step it is generating candidates and it is taking much time when the list of candidates becomes larger.

Here is the pseudo-code for Apriori Algorithm;

Algorithm 3: Apriori algorithm

F_1 = {frequent items of size 1};
for (k = 1; F_k !=ϕ; k++) do begin
    C_{k+1} = apriori-gen (F_k); // New candidates generated from F_k
    for all transactions t in database do begin
        C'_t = subset (C_{k+1},t); // Candidates contained in t
        for all candidate $c \in C'_t$ do
            $c.count++$; // Increment the count of all candidates
            in C_{k+1} that are contained in t
        end
        F_{k+1} = $\{C \in C_{k+1} | c.count \geq$ minimum suport$\}$
        //Candidates in C_{k+1} with minimum support
    end
end
Answer $\bigcup_k F_k$;

What is Frequent Pattern Mining?

Frequent Pattern Mining (AKA Association Rule Mining) is an analytical process that finds frequent patterns, associations, or causal structures from data sets found in various kinds of databases such as relational databases, transactional databases, and other data repositories. Given a set of transactions, this process aims to find the rules that enable us to predict the occurrence of a specific item based on the occurrence of other items in the transaction.

Let's look at an example of Frequent Pattern Mining. First, we will want to understand the terminology used in this type of analysis. While there are numerous metrics and factors used in this technique, for this example, we will only consider two factors namely, Support and Confidence.

Support: The support of a rule x -> y (where x and y are each items/events etc.) is defined as the proportion of transactions in the data set which contain the item set x as well as y. So, Support (x -> y)= no. of transactions which contain the item set x & y / total no. of transactions.

Confidence: The confidence of a rule x -> y is defined as: Support (x -> y) / support (x). So, it is the ratio of the number of transactions that include all items in the consequent (y in this case), as

well as the antecedent (x in this case) to the number of transactions that include all items in the antecedent (x in this case).

In the table below, Support (milk->bread) = 0.4 means milk and bread are purchased together occur in 40% of all transactions. Confidence (milk->bread) = 0.5 means that if there are 100 transactions containing milk then there will be 50 that will also contain bread.

TID	Milk	Bread	Butter	Beer
1	1	0	1	1
2	1	1	1	0
3	0	1	1	0
4	1	0	0	1
5	1	1	1	1

How Does Frequent Pattern Mining Support Business Analysis?

This method of analysis can be useful in evaluating data for various business functions and industries.

- **Basket Data Analysis**: To analyze the association of purchased items in a single basket or single purchase.
- **Cross Marketing and Selling**: To work with other businesses that complement your own, not competitors. For example, vehicle dealerships and manufacturers have cross marketing campaigns with oil and gas companies for obvious reasons.
- **Catalog Design**: The selection of items in a business' catalog are often designed to complement each other, so that buying one item will lead to buying another, so these items are often complements or closely related.
- **Medical Treatments**: Each patient is represented as a transaction containing the ordered set of diseases, and which diseases are likely to occur simultaneously/sequentially can be predicted.

To understand the value of this applied technique, let's consider two business use cases.

Use Case One

Business Problem: A retail store manager wants to conduct Market Basket analysis to come up with a better strategy of product placement and product bundling.

Business Benefit: Based on the rules generated, the store manager can strategically place the products together or in sequence leading to growth in sales and, in turn, revenue of the store. Offers such as “Buy this and get this free” or “Buy this and get % off on this” can be designed based on the rules generated.

Use Case Two

Business Problem: A bank-marketing manager wishes to analyze which products are frequently and sequentially bought together. Each customer is represented as a transaction, containing the ordered set of products, and which products are likely to be purchased simultaneously/sequentially can then be predicted.

Business Benefit: Based on the rules generated, banking products can be cross-sold to each existing or prospective customer to drive sales and bank revenue. For example, if savings, personal loan and credit cards are frequently/sequentially bought, then a new saving account customer can be cross-sold with a personal loan and credit card.

Frequent Pattern Mining (AKA Association Rule Mining) is an analytical process that finds frequent patterns, associations, or causal structures from data sets found in various kinds of data repositories. This method of analysis can be useful in evaluating data for various business functions and industries and is useful in determining the frequent patterns in buying behavior for various products and services, and in analyzing the relationships among various data points to cross-sell and bundle products, and service offerings, and to understand target audiences.

Data mining is the process of finding useful new correlations, patterns, and trends by transferring through a high amount of data saved in repositories, using pattern recognition technologies including statistical and mathematical techniques. It is the analysis of factual datasets to discover unsuspected relationships and to summarize the records in novel methods that are both logical and helpful to the data owner.

There are various concepts of data mining which are as follows −

Classification − Classification is the procedure of discovering a model that represents and distinguishes data classes or concepts, for the objective of being able to use the model to predict the class of objects whose class label is anonymous. The derived model is based on the analysis of a group of training records (i.e., data objects whose class label is familiar).

Predictions − Prediction is the same as classification, except that for prediction, the results are misrepresented in the future.

Examples of prediction functions in business and research include −

- It can be predicting the value of a stock three months into the future.
- It can be predicting the percentage increase in traffic deaths next year if the speed limit is raised.
- It can be predicting the winner of this fall's baseball World Series, based on a correspondence of team statistics.
- It can be predicted whether a definite molecule in drug discovery will begin a cost-effective new drug for a pharmaceutical company.

Association Rules and Recommendation Systems − Association rules, or affinity analysis, are designed to find such general associations patterns between items in large databases. The rules can be used in several methods. For example, grocery stores can use such information for product placement.

They can use the rules for weekly promotional offers or for bundling products. Association rules derived from a hospital database on patients' symptoms during consecutive hospitalizations can help find "which symptom is followed by what other symptom" to help predict future symptoms for returning patients.

Data Reduction − Data mining is used to the selected data in a huge amount database. When data analysis and mining is completed on a huge amount of records then it takes a very high time to process, which develops it impossible and infeasible.

It can reduce the processing time for data analysis, data reduction techniques are used to obtain a reduced representation of the dataset that is much smaller in volume by maintaining the integrity of the original data. By reducing the data, the efficiency of the data mining process is improved which produces the same analytical results.

Data reduction aims to define it more compactly. When the data size is smaller, it is easier to use mature and computationally high-cost algorithms. The reduction of the data may be in terms of the number of rows (records) or terms of the number of columns (dimensions).

Frequent Item set in Data set (Association Rule Mining)

Association Mining searches for frequent items in the data-set. In frequent mining usually the interesting associations and correlations between item sets in transactional and relational databases are found. In short, Frequent Mining shows which items appear together in a transaction or relation. **Need of Association Mining:** Frequent mining is generation of association rules from a Transactional Dataset. If there are 2 items X and Y purchased frequently then its good to put them together in stores or provide some discount offer on one item on purchase of other item. This can really increase the sales. For example it is likely to find that if a customer buys **Milk** and **bread** he/she also buys **Butter**. So the association rule is **['milk]^['bread']=>['butter']**. So seller can suggest the customer to buy butter if he/she buys Milk and Bread.

Important Definitions:

- **Support:** It is one of the measures of interestingness. This tells about usefulness and certainty of rules. **5% Support** means total 5% of transactions in database follow the rule.

Support (A -> B) = Support count (A ∪ B)

- **Confidence:** A confidence of 60% means that 60% of the customers who purchased a milk and bread also bought butter.

Confidence (A -> B) = Support count (A ∪ B) / Support count (A)
If a rule satisfies both minimum support and minimum confidence, it is a strong rule.

- **Support count(X)**: Number of transactions in which X appears. If X is a **union** B then it is the number of transactions in which A and B both are present.
- **Maximal Item set:** An item set is maximal frequent if none of its supersets are frequent.
- **Closed Item set:** An item set is closed if none of its immediate supersets have same support count same as Item set.
- **K- Itemset:** Itemset which contains K items is a K-item set. So it can be said that an item set is frequent if the corresponding support count is greater than minimum support count.

Example On finding Frequent Item sets – Consider the given dataset with given transactions.

1	{A,C,D}
2	{B,C,D}
3	{A,B,C,D}
4	{B,D}
5	{A,B,C,D}

- Let's say minimum support count is 3
- Relation hold is maximal frequent => closed => frequent

1-frequent: *{A} = 3; // not closed due to {A, C} and not maximal {B} = 4; // not closed due to {B, D} and no maximal {C} = 4; // not closed due to {C, D} not maximal {D} = 5; // closed item-set since not immediate super-set has same count. Not maximal*
2-frequent: *{A, B} = 2 // not frequent because support count < minimum support count so ignore {A, C} = 3 // not closed due to {A, C, D} {A, D} = 3 // not closed due to {A, C, D} {B, C} = 3 //*

not closed due to {B, C, D} {B, D} = 4 // closed but not maximal due to {B, C, D} {C, D} = 4 // closed but not maximal due to {B, C, D}
3-frequent: *{A, B, C} = 2 // ignore not frequent because support count < minimum support count {A, B, D} = 2 // ignore not frequent because support count < minimum support count {A, C, D} = 3 // maximal frequent {B, C, D} = 3 // maximal frequent*
4-frequent: *{A, B, C, D} = 2 //ignore not frequent </*

Apriori Algorithm

Apriori algorithm is given by R. Agrawal and R. Srikant in 1994 for finding frequent item sets in a dataset for Boolean association rule. Name of the algorithm is Apriori because it uses prior knowledge of frequent item set properties. We apply an iterative approach or level-wise search where k-frequent item sets are used to find k+1 item sets.

To improve the efficiency of level-wise generation of frequent item sets, an important property is used called *Apriori property* which helps by reducing the search space.

Apriori Property –

All non-empty subset of frequent item set must be frequent. The key concept of Apriori algorithm is its anti-monotonicity of support measure. Apriori assumes that

All subsets of a frequent item set must be frequent (Apriori property).
If an item set is infrequent, all its supersets will be infrequent.

Before we start understanding the algorithm, go through some definitions which are explained in my previous post.
Consider the following dataset and we will find frequent item sets and generate association rules for them.

TID	items
T1	I1, I2 , I5
T2	I2,I4
T3	I2,I3
T4	I1,I2,I4
T5	I1,I3
T6	I2,I3
T7	I1,I3
T8	I1,I2,I3,I5
T9	I1,I2,I3

Minimum support count is 2
minimum confidence is 60%
Step-1: K=1
(I) Create a table containing support count of each item present in dataset – Called **C1 (candidate set)**

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

(II) compare candidate set item's support count with minimum support count(here min_support=2 if support count of candidate set items is less than min_support then remove those items). This gives us item set L1.

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

Step-2: K=2

- Generate candidate set C2 using L1 (this is called join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common.
- Check all subsets of an item set are frequent or not and if not frequent remove that item set.(Example subset of{I1, I2} are {I1}, {I2} they are frequent. Check for each item set)
- Now find support count of these item sets by searching in dataset.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I4	1
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I3,I4	0
I3,I5	1
I4,I5	0

(II) compare candidate (C2) support count with minimum support count(here min_support=2 if support count of candidate set item is less than min_support then remove those items) this gives us item set L2.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I2,I5	2

Step-3:

- Generate candidate set C3 using L2 (join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common. So here, for L2, first element should match.
  So item set generated by joining L2 is {I1, I2, I3}{I1, I2, I5}{I1, I3, i5}{I2, I3, I4}{I2, I4, I5}{I2, I3, I5}
- Check if all subsets of these item sets are frequent or not and if not, then remove that item set. (Here subset of {I1, I2, I3} are {I1, I2},{I2, I3},{I1, I3} which are frequent. For {I2, I3, and I4}, subset {I3, I4} is not frequent so remove it. Similarly check for every item set)
- find support count of these remaining item set by searching in dataset.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

(II) Compare candidate (C3) support count with minimum support count(here min_support=2 if support count of candidate set item is less than min_support then remove those items) this gives us item set L3.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

Step-4:

- Generate candidate set C4 using L3 (join step). Condition of joining L_{k-1} and L_{k-1} (K=4) is that, they should have (K-2) elements in common. So here, for L3, first 2 elements (items) should match.
- Check all subsets of these item sets are frequent or not (Here item set formed by joining L3 is {I1, I2, I3, I5} so its subset contains {I1, I3, I5}, which is not frequent). So no item set in C4
- We stop here because no frequent item sets are found further

Thus, we have discovered all the frequent item-sets. Now generation of strong association rule comes into picture. For that we need to calculate confidence of each rule.

Confidence –

A confidence of 60% means that 60% of the customers, who purchased milk and bread also bought butter.
Confidence (A->B)=Support count(A∪B)/Support count(A)
So here, by taking an example of any frequent item set, we will show the rule generation.
Item set {I1, I2, I3} //from L3
SO rules can be
[I1^I2]=>[I3] //confidence = sup(I1^I2^I3)/sup(I1^I2) = 2/4*100=50%
[I1^I3]=>[I2] //confidence = sup(I1^I2^I3)/sup(I1^I3) = 2/4*100=50%
[I2^I3]=>[I1] //confidence = sup(I1^I2^I3)/sup(I2^I3) = 2/4*100=50%
[I1]=>[I2^I3] //confidence = sup(I1^I2^I3)/sup(I1) = 2/6*100=33%
[I2]=>[I1^I3] //confidence = sup(I1^I2^I3)/sup(I2) = 2/7*100=28%
[I3]=>[I1^I2] //confidence = sup(I1^I2^I3)/sup(I3) = 2/6*100=33%

So if minimum confidence is 50%, then first 3 rules can be considered as strong association rules.

Limitations of Apriori Algorithm

Apriori Algorithm can be slow. The main limitation is time required to hold a vast number of candidate sets with much frequent item sets, low minimum support or large item sets i.e. it is not an efficient approach for large number of datasets. For example, if there are 10^4 from frequent 1- item sets, it need to generate more than 10^7 candidates into 2-length which in turn they will be tested and accumulate. Furthermore, to detect frequent pattern in size 100 i.e. v1, v2… v100, it have to generate 2^100 candidate item sets that yield on costly and wasting of time of candidate generation. So, it will check for many sets from candidate item sets, also it will scan database many times repeatedly for finding candidate item sets. Apriori will be very low and inefficiency when memory capacity is limited with large number of transactions.

Apriori Algorithm – Frequent Pattern Algorithms

Apriori algorithm was the first algorithm that was proposed for frequent item set mining. It was later improved by R Agarwal and R Srikant and came to be known as Apriori. This algorithm uses two steps “join” and “prune” to reduce the search space. It is an iterative approach to discover the most frequent item sets.

Apriori says:

The probability that item I is not frequent is if:

- P (I) < minimum support threshold, then I is not frequent.
- P (I+A) < minimum support threshold, then I+A is not frequent, where A also belongs to item set.
- If an item set has value less than minimum support then all of its supersets will also fall below min support, and thus can be ignored. This property is called the Ant monotone property.

The steps followed in the Apriori Algorithm of data mining are:

1. **Join Step**: This step generates (K+1) item set from K-item sets by joining each item with itself.
2. **Prune Step**: This step scans the count of each item in the database. If the candidate item does not meet minimum support, then it is regarded as infrequent and thus it is removed. This step is performed to reduce the size of the candidate item sets.

Steps in Apriori

Apriori algorithm is a sequence of steps to be followed to find the most frequent item set in the given database. This data mining technique follows the join and the prune steps iteratively until the most frequent item set is achieved. A minimum support threshold is given in the problem or it is assumed by the user.

#1) In the first iteration of the algorithm, each item is taken as a 1-itemsets candidate. The algorithm will count the occurrences of each item.
#2) Let there be some minimum support, min_sup (egg 2). The set of 1 – item sets whose occurrence is satisfying the min sup are determined. Only those candidates which count more than or equal to min_sup, are taken ahead for the next iteration and the others are pruned.
#3) Next, 2-itemset frequent items with min_sup are discovered. For this in the join step, the 2-itemset is generated by forming a group of 2 by combining items with itself.
#4) The 2-itemset candidates are pruned using min-sup threshold value. Now the table will have 2 –item sets with min-sup only.
#5) The next iteration will form 3 –item sets using join and prune step. This iteration will follow ant monotone property where the subsets of 3-itemsets that is the 2 –item set subsets of each group fall in min_sup. If all 2-itemset subsets are frequent then the superset will be frequent otherwise it is pruned.
#6) next steps will follow making 4-itemset by joining 3-itemset with itself and pruning if its subset does not meet the min_sup criteria. The algorithm is stopped when the most frequent item set is achieved.

Example of Apriori: **Support threshold=50%, Confidence= 60%**

TABLE-1

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

Solution:

Support threshold=50% => 0.5*6= 3 => min_sup=3

1. Count of Each Item

TABLE-2

Item	Count
I1	4
I2	5
I3	4
I4	4
I5	2

2. Prune Step: TABLE -2 shows that I5 item does not meet min_sup=3, thus it is deleted; only I1, I2, I3, and I4 meet min_sup count.

TABLE-3

Item	Count
I1	4
I2	5
I3	4
I4	4

3. Join Step: Form 2-itemset. From **TABLE-1** find out the occurrences of 2-itemset.

TABLE-4

Item	Count
I1,I2	4
I1,I3	3
I1,I4	2
I2,I3	4
I2,I4	3
I3,I4	2

4. Prune Step: TABLE -4 shows that item set {I1, I4} and {I3, I4} does not meet min_sup, thus it is deleted.

TABLE-5

Item	Count
I1,I2	4
I1,I3	3
I2,I3	4
I2,I4	3

5. Join and Prune Step: Form 3-itemset. From the **TABLE- 1** find out occurrences of 3-itemset. From **TABLE-5**, find out the 2-itemset subsets which support min_sup.

We can see for item set {I1, I2, I3} subsets, {I1, I2}, {I1, I3}, {I2, I3} are occurring in **TABLE-5** thus {I1, I2, I3} is frequent.

We can see for item set {I1, I2, I4} subsets, {I1, I2}, {I1, I4}, {I2, I4}, {I1, I4} is not frequent, as it is not occurring in **TABLE-5** thus {I1, I2, I4} is not frequent, hence it is deleted.

TABLE-6

Item
I1,I2,I3
I1,I2,I4
I1,I3,I4
I2,I3,I4

Only {I1, I2, I3} is frequent.

6. Generate Association Rules: From the frequent item set discovered above the association could be:

{I1, I2} => {I3}

Confidence = support {I1, I2, I3} / support {I1, I2} = (3/ 4)* 100 = 75%

{I1, I3} => {I2}

Confidence = support {I1, I2, I3} / support {I1, I3} = (3/ 3)* 100 = 100%

{I2, I3} => {I1}

Confidence = support {I1, I2, I3} / support {I2, I3} = (3/ 4)* 100 = 75%

{I1} => {I2, I3}

Confidence = support {I1, I2, I3} / support {I1} = (3/ 4)* 100 = 75%

{I2} => {I1, I3}

Confidence = support {I1, I2, I3} / support {I2 = (3/ 5)* 100 = 60%

{I3} => {I1, I2}

Confidence = support {I1, I2, I3} / support {I3} = (3/ 4)* 100 = 75%

This shows that all the above association rules are strong if minimum confidence threshold is 60%.

The Apriori Algorithm: Pseudo Code
C: Candidate item set of size k

L: Frequent item set of size k

- Join Step: C_k is generated by joining L_{k-1} with itself
- Prune Step: Any (k-1)-itemset that is not frequent cannot be a subset of a frequent k-itemset
- Pseudo-code :C_k: Candidate itemset of size k
  L_k: frequent itemset of size k

```
L1 = {frequent items};
for (k = 1; Lk !=∅; k++) do begin
   Ck+1 = candidates generated from Lk;
   for each transaction t in database do
         increment the count of all candidates in Ck+1
         that are contained in t
   Lk+1 = candidates in Ck+1 with min_support
   end
return ∪k Lk;
```

[image source]

Advantages

1. Easy to understand algorithm
2. Join and Prune steps are easy to implement on large item sets in large databases

Disadvantages

1. It requires high computation if the item sets are very large and the minimum support is kept very low.
2. The entire database needs to be scanned.

Methods to Improve Apriori Efficiency

Many methods are available for improving the efficiency of the algorithm.

1. **Hash-Based Technique:** This method uses a hash-based structure called a hash table for generating the k-item sets and its corresponding count. It uses a hash function for generating the table.
2. **Transaction Reduction:** This method reduces the number of transactions scanning in iterations. The transactions which do not contain frequent items are marked or removed.
3. **Partitioning:** This method requires only two database scans to mine the frequent item sets. It says that for any item set to be potentially frequent in the database, it should be frequent in at least one of the partitions of the database.
4. **Sampling:** This method picks a random sample S from Database D and then searches for frequent item set in S. It may be possible to lose a global frequent item set. This can be reduced by lowering the min_sup.
5. **Dynamic Item set Counting:** This technique can add new candidate item sets at any marked start point of the database during the scanning of the database.

Applications of Apriori Algorithm

Some fields where Apriori is used:

1. **In Education Field:** Extracting association rules in data mining of admitted students through characteristics and specialties.
2. **In the Medical field:** For example Analysis of the patient's database.
3. **In Forestry:** Analysis of probability and intensity of forest fire with the forest fire data.
4. Apriori is used by many companies like Amazon in the **Recommender System** and by Google for the auto-complete feature.

Conclusion

Apriori algorithm is an efficient algorithm that scans the database only once.

It reduces the size of the item sets in the database considerably providing a good performance. Thus, data mining helps consumers and industries better in the decision-making process.

Frequent Pattern Growth Algorithm

This algorithm is an improvement to the Apriori method. A frequent pattern is generated without the need for candidate generation. FP growth algorithm re

Represents the database in the form of a tree called a frequent pattern tree or FP tree.

This tree structure will maintain the association between the item sets. The database is fragmented using one frequent item. This fragmented part is called “pattern fragment”. The item sets of these fragmented patterns are analyzed. Thus with this method, the search for frequent item sets is reduced comparatively.

FP Tree

Frequent Pattern Tree is a tree-like structure that is made with the initial item sets of the database. The purpose of the FP tree is to mine the most frequent pattern. Each node of the FP tree represents an item of the item set.

The root node represents null while the lower nodes represent the item sets. The association of the nodes with the lower nodes that is the item sets with the other item sets are maintained while forming the tree.

Frequent Pattern Algorithm Steps
The frequent pattern growth method lets us find the frequent pattern without candidate generation.

Let us see the steps followed to mine the frequent pattern using frequent pattern growth algorithm:

#1) The first step is to scan the database to find the occurrences of the item sets in the database. This step is the same as the first step of Apriori. The count of 1-itemsets in the database is called support count or frequency of 1-itemset.
#2) The second step is to construct the FP tree. For this, create the root of the tree. The root is represented by null.
#3) The next step is to scan the database again and examine the transactions. Examine the first transaction and find out the item set in it. The item set with the max count is taken at the top, the next item set with lower count and so on. It means that the branch of the tree is constructed with transaction item sets in descending order of count.
#4) The next transaction in the database is examined. The item sets are ordered in descending order of count. If any item set of this transaction is already present in another branch (for example in the 1st transaction), then this transaction branch would share a common prefix to the root.

This means that the common item set is linked to the new node of another item set in this transaction.

#5) Also, the count of the item set is incremented as it occurs in the transactions. Both the common node and new node count is increased by 1 as they are created and linked according to transactions.
#6) The next step is to mine the created FP Tree. For this, the lowest node is examined first along with the links of the lowest nodes. The lowest node represents the frequency pattern length 1. From this, traverse the path in the FP Tree. This path or paths are called a conditional pattern base.
Conditional pattern base is a sub-database consisting of prefix paths in the FP tree occurring with the lowest node (suffix).

#7) Construct a Conditional FP Tree, which is formed by a count of item sets in the path. The item sets meeting the threshold support are considered in the Conditional FP Tree.

#8) Frequent Patterns are generated from the Conditional FP Tree.

Example of FP-Growth Algorithm
Support threshold=50%, Confidence= 60%
Table 1

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

Solution:
Support threshold=50% => 0.5*6= 3 => min_sup=3

1. Count of each item
Table 2

Item	Count
I1	4
I2	5

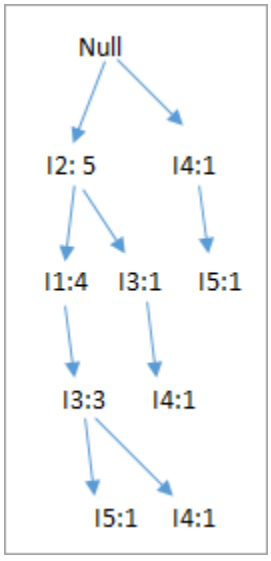
Item	Count
I3	4
I4	4
I5	2

2. Sort the item set in descen ding order.
Table 3

Item	Count
I2	5
I1	4
I3	4
I4	4

3. Build FP Tree

1. Considering the root node null.
2. The first scan of Transaction T1: I1, I2, I3 contains three items {I1:1}, {I2:1}, {I3:1}, where I2 is linked as a child to root, I1 is linked to I2 and I3 is linked to I1.
3. T2: I2, I3, I4 contains I2, I3, and I4, where I2 is linked to root, I3 is linked to I2 and I4 is linked to I3. But this branch would share I2 node as common as it is already used in T1.
4. Increment the count of I2 by 1 and I3 is linked as a child to I2, I4 is linked as a child to I3. The count is {I2:2}, {I3:1}, {I4:1}.
5. T3: I4, I5. Similarly, a new branch with I5 is linked to I4 as a child is created.
6. T4: I1, I2, I4. The sequence will be I2, I1, and I4. I2 is already linked to the root node; hence it will be incremented by 1. Similarly I1 will be incremented by 1 as it is already linked with I2 in T1, thus {I2:3}, {I1:2}, {I4:1}.
7. T5:I1, I2, I3, I5. The sequence will be I2, I1, I3, and I5. Thus {I2:4}, {I1:3}, {I3:2}, {I5:1}.
8. T6: I1, I2, I3, I4. The sequence will be I2, I1, I3, and I4. Thus {I2:5}, {I1:4}, {I3:3}, {I4 1}.

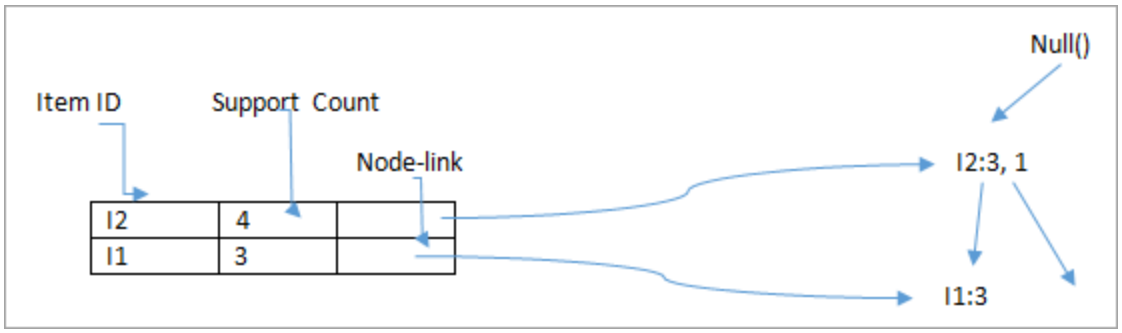


4. Mining of FP-tree is summarized below:

1. The lowest node item I5 is not considered as it does not have a min support count, hence it is deleted.
2. The next lower node is I4. I4 occurs in 2 branches , {I2,I1,I3:,I41},{I2,I3,I4:1}. Therefore considering I4 as suffix the prefix paths will be {I2, I1, I3:1}, {I2, I3: 1}. this forms the conditional pattern base.
3. The conditional pattern base is considered a transaction database; an FP-tree is constructed. This will contain {I2:2, I3:2}, I1 is not considered as it does not meet the min support count.
4. This path will generate all combinations of frequent patterns : {I2,I4:2},{I3,I4:2},{I2,I3,I4:2}
5. For I3, the prefix path would be: {I2,I1:3},{I2:1}, this will generate a 2 node FP-tree : {I2:4, I1:3} and frequent patterns are generated: {I2,I3:4}, {I1:I3:3}, {I2,I1,I3:3}.
6. For I1, the prefix path would be: {I2:4} this will generate a single node FP-tree: {I2:4} and frequent patterns are generated: {I2, I1:4}.

Item	Conditional Pattern Base	Conditional FP-tree	Frequent Patterns Generated
I4	{I2,I1,I3:1},{I2,I3:1}	{I2:2, I3:2}	{I2,I4:2},{I3,I4:2},{I2,I3,I4:2
I3	{I2,I1:3},{I2:1}	{I2:4, I1:3}	{I2,I3:4}, {I1:I3:3}, {I2,I1,I3:3}
I1	{I2:4}	{I2:4}	{I2,I1:4}

The dia ram given below depicts the conc itional FP tree associated with the conditional node I3.



Advantages of FP Growth Algorithm

1. This algorithm needs to scan the database only twice when compared to Apriori which scans the transactions for each iteration.
2. The pairing of items is not done in this algorithm and this makes it faster.
3. The database is stored in a compact version in memory.
4. It is efficient and scalable for mining both long and short frequent patterns.

Disadvantages of FP-Growth Algorithm

1. FP Tree is more cumbersome and difficult to build than Apriori.
2. It may be expensive.
3. When the database is large, the algorithm may not fit in the shared memory.

FP Growth vs. Apriori

FP Growth	Apriori
Pattern Generation	
FP growth generates pattern by constructing a FP tree	Apriori generates pattern by pairing the items into singletons, pairs and triplets.
Candidate Generation	
There is no candidate generation	Apriori uses candidate generation
Process	
The process is faster as compared to Apriori. The runtime of process increases linearly with increase in number of item sets.	The process is comparatively slower than FP Growth, the runtime increases exponentially with increase in number of item sets
Memory Usage	
A compact version of database is saved	The candidates combinations are saved in memory

ECLAT

The above method, Apriori and FP growth, mine frequent item sets using horizontal data format. ECLAT is a method of mining frequent item sets using the vertical data format. It will transform the data in the horizontal data format into the vertical format.

For Example, Apriori and FP growth use:

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

The ECLAT will have the format of the table as:

Item	Transaction Set
I1	{T1,T4,T5,T6}
I2	{T1,T2,T4,T5,T6}
I3	{T1,T2,T5,T6}
I4	{T2,T3,T4,T5}
I5	{T3,T5}

This method will form 2-itemsets, 3 item sets, k item sets in the vertical data format. This process with k is incr
techniques such as dif used are used along with Apriori.

This method has an ad antage over Apriori as it does not require scanning the database to find the support of k+1 ite n sets. This is because the Transaction set will carry the count of occurrence of each ite m in the transaction (support). The bottleneck comes when there are many transactions taking hug

Conclusion

The Apriori algorithm is used for mining association rules. It works on the principle, “the non-empty subsets of frequ
(k-1) item sets and scans the database to find the frequent item sets.

Frequent Pattern Grow
generation. It construc

The focus of the FP Growth algorithm is on fragmenting the paths of the items and mining frequent patterns.

ML | Frequent Pattern Growth Algorithm

The two primary drawbacks of the Apriori Algorithm are:

1. At each step, candidate sets have to be built.
2. To build the candidate sets, the algorithm has to repeatedly scan the database.

These two properties inevitably make the algorithm slower. To overcome these redundant steps, a new association-rule mining algorithm was developed named Frequent Pattern Growth Algorithm. It overcomes the disadvantages of the Apriori algorithm by storing all the transactions in a Tire Data Structure. Consider the following data:-

The above-given data is a hypothetical dataset of transactions with each letter representing an item. The frequency of each individual item is computed:-

Let the minimum support be 3. A **Frequent Pattern set** is built which will contain all the elements whose frequency is greater than or equal to the minimum support. These elements are stored in descending order of their respective frequencies. After insertion of the relevant items, the set L looks like this:-

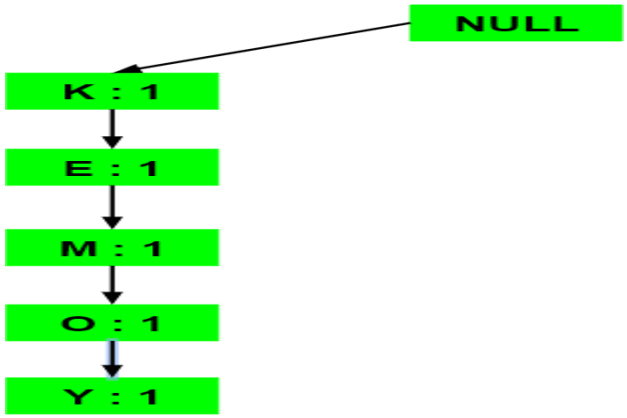
L = {K: 5, E: 4, M: 3, O: 3, Y: 3}

Now, for each transaction, the respective **Ordered-Item set** is built. It is done by iterating the Frequent Pattern set and checking if the current item is contained in the transaction in question. If the current item is contained, the item is inserted in the Ordered-Item set for the current transaction. The following table is built for all the transactions:

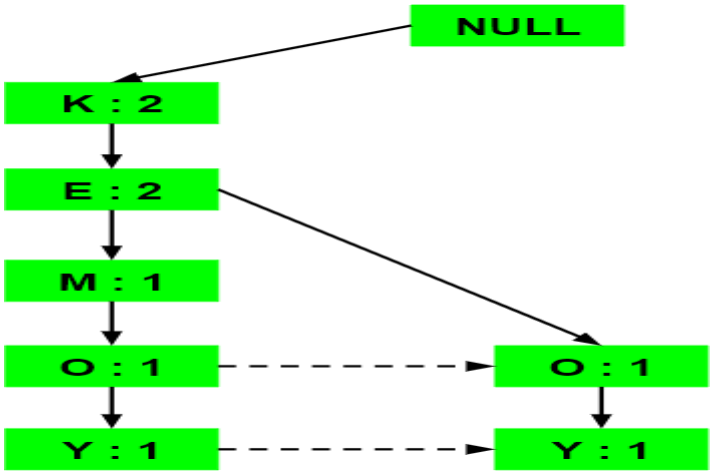
Now, all the Ordered-Item sets are inserted into a Tire Data Structure.

a) **Inserting the set {K, E, M, O, and Y}:**

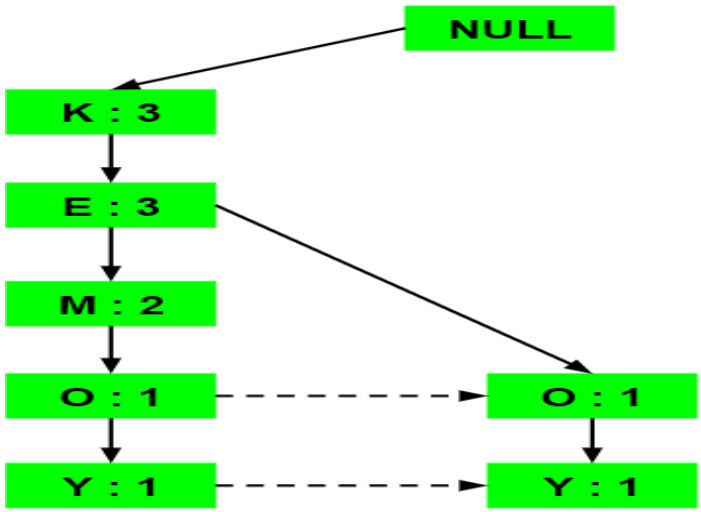
Here, all the items are simply linked one after the other in the order of occurrence in the set and initialize the support count for each item as 1.



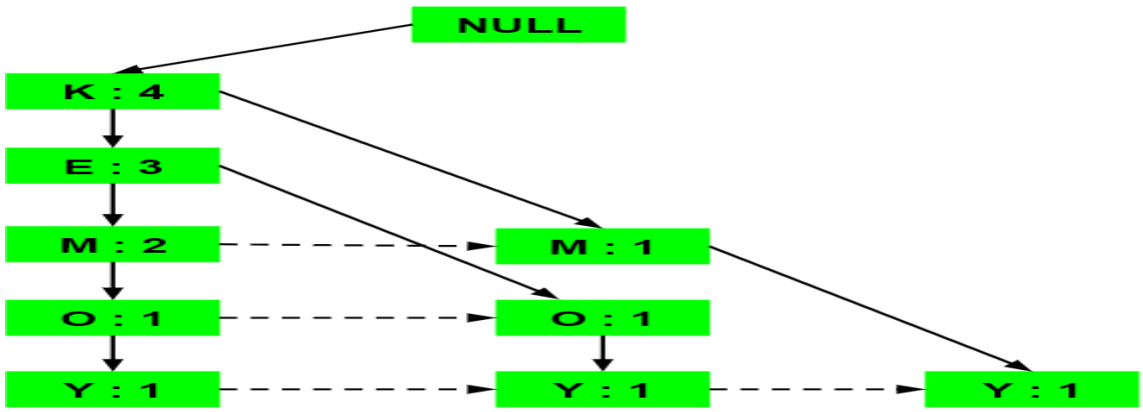
b) **Inserting the set {K, E, O, and Y}:**
Till the insertion of the elements K and E, simply the support count is increased by 1. On inserting O we can see that there is no direct link between E and O, therefore a new node for the item O is initialized with the support count as 1 and item E is linked to this new node. On inserting Y, we first initialize a new node for the item Y with support count as 1 and link the new node of O with the new node of Y.



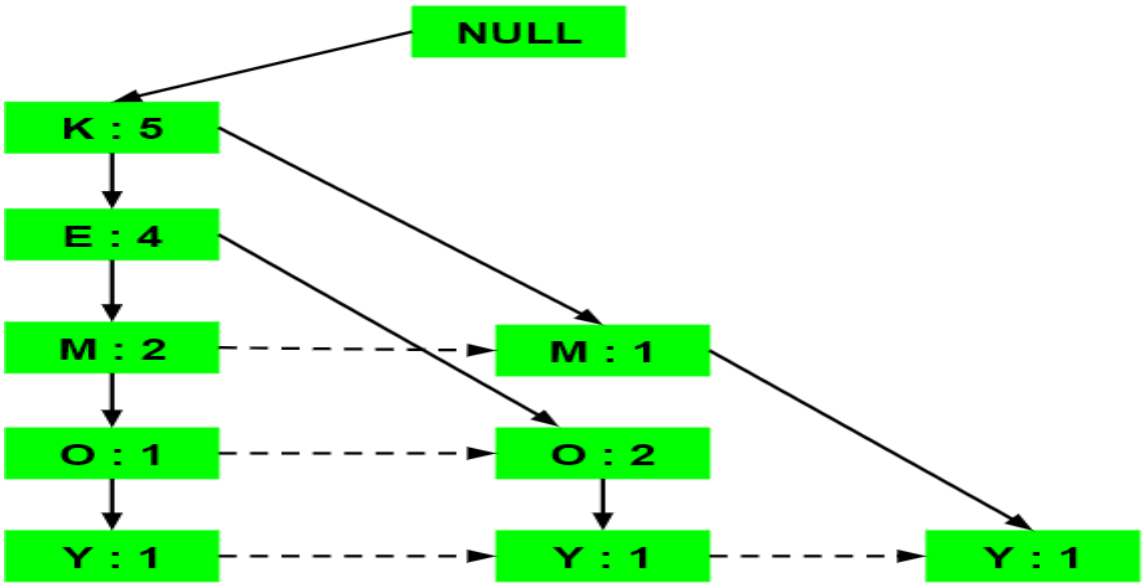
c) **Inserting the set {K, E, M}:**
Here simply the support count of each element is increased by 1.



d) **Inserting the set {K, M, Y}:**
Similar to step b), first the support count of K is increased, then new nodes for M and Y are initialized and linked accordingly.



e) **Inserting the set {K, E, O}:**
Here simply the support counts of the respective elements are increased. Note that the support count of the new node of item O is increased.



Now, for each item, the **Conditional Pattern Base** is computed which is path labels of all the paths which lead to any node of the given item in the frequent-pattern tree. Note that the items in the below table are arranged in the ascending order of their frequencies.

Items	Conditional Pattern Base
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}
O	{{K,E,M : 1}, {K,E : 2}}
M	{{K,E : 2}, {K : 1}}
E	{K : 4}
K	

Now for each item, the **Conditional Frequent Pattern Tree is built.** It is done by taking the set of elements that is common in all the paths in the Conditional Pattern Base of that item and calculating its support count by summing the support counts of all the paths in the Conditional Pattern Base.

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

From the Conditional Frequent Pattern tree, the **Frequent Pattern rules** are generated by pairing the items of the Conditional Frequent Pattern Tree set to the corresponding to the item as given in the below table.

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 4>}
K	

For each row, two types of association rules can be inferred for example for the first row which contains the element, the rules K -> Y and Y -> K can be inferred. To determine the valid rule, the confidence of both the rules is calculated and the one with confidence greater than or equal to the minimum confidence value is retained.

Close and Max Patterns

Compress frequent patterns using closed patterns and max patterns

Transaction-id	Items bought
10	A, B, D
20	A, C, D
30	A, D, E
40	B, E, F
50	B, C, D, E, F

When we have many items, the total number of itemsets grows exponentially to the number of items in the database: |item set| = 2n. This immediately causes both space and time complexity problems in finding frequent patterns.

For example, the table above contains five transaction records and six items. This small database can have 26 = 64 possible item set. To deal with this problem, we now define a series of sets.

Sub-pattern (proper subset)

To find efficient algorithms for mining frequent patterns, we will start with two basic concepts of patterns. A pattern is an item set, therefore the order of the items are not important. What will be useful is the properties of subsets of supersets of frequent patterns.

Let us start with subsets. We call any proper subset of a pattern, a sub-pattern. Here is an example,

- If a pattern X contains, 3 items: X= {a, b, c}, any proper subsets, such as sets containing a, or b, or c, or ab, or ac, or BC, are all sub patterns.

Now, suppose we have a pattern with 100 items. How many sub-patterns can we find for this pattern?

- A lot, it is more than 10 to the power of 30.

That means if this pattern is a frequent pattern, there are 10 to the power of 30 frequent patterns.

Super-pattern (proper superset)

Another concept is of course super-pattern. A super pattern of a pattern is a proper super set of the pattern.

Here are some examples of the pattern containing the 3 items: a, b, c

- abed, abce, abcde, …

Close Pattern

Now, we are ready for two important concepts of frequent patterns. Let's start with closed patterns.

A **closed pattern** is

1. a frequent pattern. So it meets the minimum support criteria.
2. In addition to that, all super-patterns of a closed pattern are less frequent than the closed pattern.

Formally, this is defined as:

CP is a **Closed Pattern** if
- CP is a **frequent pattern** and
- All super-patterns of CP are less frequent than CP
  - For all super-patterns S of CP,
    Supp(S) < Supp(CP)

Let's see some examples.

1. Suppose, the minimum support count is 2.
2. For the first example, suppose there are a total of 3 items: a, b, c.
3. Suppose a pattern ab has support count of 2 and a pattern abc has support count of 2.
4. Is the pattern ab is a closed pattern?

Pattern ab is a frequent pattern, but it has a super-pattern that is NOT less frequent than ab.

For the second example,

1. Suppose there are a total of 3 items: x, y, z.
2. Suppose a pattern xy has support count of 3 and a pattern xyz has support count of 2.
3. Is the pattern xy is a closed pattern?

Pattern xy is a frequent pattern and also the only super-pattern xyz is less frequent than xy.

Therefore, xy is a closed pattern.

Max Pattern

Next, let's define max pattern.

A **max pattern** is

1. a frequent pattern. So it also meets the minimum support criteria like closed pattern
2. In addition, but unlike closed pattern, all super-patterns of a max pattern are NOT frequent patterns.

This is formally written as

MP is a **Max Pattern** if
- MP is a frequent pattern and
- All super-patterns of MP are not frequent
  - For all super-pattern SP of MP, $Supp(SP) < Supp_{Min}$

Let's see some examples as well.

1. Suppose, the minimum support count is 2.
2. Like before, for the first example, suppose there are a total of 3 items: a, b, c.
3. Suppose a pattern ab has support count of 3 and a pattern abc has support count of 2.
4. Is the pattern ab is a max pattern?

Pattern ab is a frequent pattern, but it has a super-pattern that is a frequent pattern as well. So, pattern ab is NOT a max pattern.

For the second example,

1. Suppose there are a total of 3 items: x, y, z.
2. Suppose a pattern xy has support count of 3 and a pattern xyz has support count of 1.
3. Is the pattern xy is a max pattern?

Pattern xy is a frequent pattern and also the only super-pattern xyz is NOT a frequent pattern. Therefore, xy is a max pattern.

Data Science And Data Analysis With Python

The approaches to mining multilevel association rules are based on the support confidence framework. The top-down strategy is employed where counts are accumulated for the calculation of frequent item sets at each concept level, starting at concept level 1 and working towards the lower specific concept levels until more frequent item sets can be found using the Apriori algorithm.

Data can be generalized by replacing low-level concepts within the data with their higher-level concepts or ancestors from a concept hierarchy. In a concept hierarchy, which is represented as a tree with the root as D i.e., Task-relevant data.

The popular area of application for the multi-level association is market basket analysis, which studies the buying habits of customers by searching for sets of items that are frequently, purchased together which was displayed in the concept of concept hierarchy.

Each node indicates an item or item set that has been examined. There are various approaches for finding frequent item sets at any level of abstraction. Some of the methods which are in use are 'using uniform minimum support for all levels', using reduced minimum support at low levels, level-by-level independent.

Multi-level databases need a hierarchy-data encoded transaction table rather than the initial transaction table. This is useful when we are interested in only a portion of the transaction database such as food, instead of all the items. This way we can first collect the relevant set of data and then work repeatedly on the task-relevant set. Thus in the transaction table, each item is encoded as a sequence of digits.

Using uniform minimum support for all levels − When a uniform minimum support threshold is used, the search procedure is simplified. An optimization technique can be adopted, based on the knowledge that an ancestor is a superset of its descendants, the search avoids examining item sets containing any item whose ancestor does not have minimum support.

The main drawback of the uniform support approach is that the items at lower levels of abstraction will occur as frequently as those at higher levels of abstraction.

Using reduced minimum support at lower levels − each level of abstraction has its minimum support threshold. The lower the abstraction level, the smaller the equivalent threshold. The following search categories for mining multiple-level association with reduced support are −

- **Level by level independent** − It is a full breadth search; background knowledge of frequent item sets is used for pruning. Here each node is examined regardless of the parent node is found to be frequent.
- **Level cross-filtering by a single item** − an item as the ith level is determined if and only if its parent node at the (i-1) the level is frequent.
- **Level cross-filtering by k-item set** − An item set at the ith level is determined if and only if its equivalent parent A-item set at the (i-1)the level is frequent.